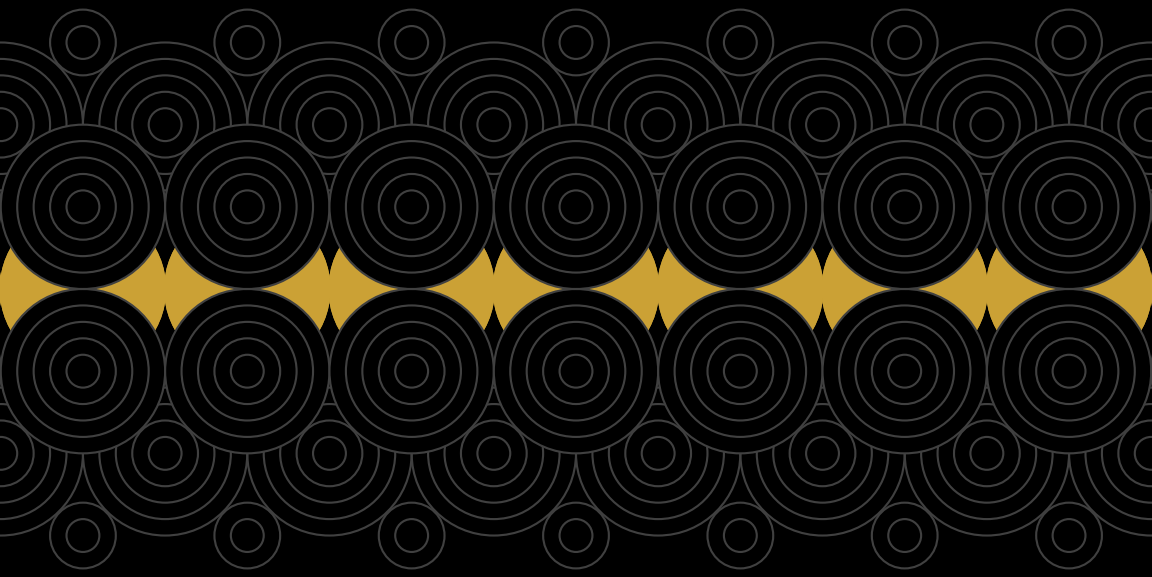
Henilsinh Raj

Fundamentals of Natural Language processing for Beginners

Your First Step into the World of NLP



Henilsinh Raj

Henilsinh Raj

Fundamentals of Natural Language processing for Beginners

Your First Step into the World of NLP

Henilsinh Raj

Preface

Natural Language Processing (NLP) has rapidly evolved into one of the most transformative fields in artificial intelligence. From search engines and chatbots to voice assistants and machine translation, NLP has become deeply embedded in our daily lives. Despite its impact, many beginners often find the subject intimidating due to its combination of linguistics, mathematics, and computer science. This book, Fundamentals of Natural Language Processing for Beginners, is written to break down these barriers. It provides a clear and structured introduction to the fundamental concepts, techniques, and applications of NLP. My goal is to guide readers step by step starting from the basics of text representation and preprocessing, progressing through classical models, and eventually exploring modern deep learning approaches that power today's state-of-the-art systems.

My journey in artificial intelligence began with research in the healthcare domain, where I published papers on convolutional neural networks (CNNs) for tumor and chest disease detection, hybrid deep learning models for toxicity classification, and applications of architectures like VGG-19 and BERT. These experiences not only deepened my understanding of deep learning but also strengthened my belief in the importance of accessible resources for beginners. As my research progressed, I extended my focus to Natural Language Processing itself. I have authored papers on developing a Hindi tokenizer, creating a language generation model for Sanskrit, and conducting systematic reviews on digital reading

platforms. These works allowed me to explore the intersection of computational linguistics and deep learning, while also addressing challenges unique to low-resource and classical languages. Building on these experiences, I authored my first book, Fundamentals of Convolutional Neural Network, which received recognition as a practical, beginner-friendly guide to CNNs. That project encouraged me to expand further into another domain that fascinated me how machines understand human language. This book is a continuation of that journey, written to provide a solid foundation in NLP for those just starting out. I have designed this book for students, self-learners, and aspiring researchers who wish to build a practical understanding of NLP without feeling overwhelmed. Each chapter is supplemented with intuitive explanations, real-world examples, and insights from both classical methods and modern neural approaches. Writing this book has been deeply personal to me. My motivation comes from my own early struggles with understanding the subject, as well as my desire to create resources that make complex ideas simple. I hope this work serves as a companion for your journey into the world of NLP and inspires you to explore, experiment, and contribute to this exciting domain.

Finally, I would like to express my gratitude to my mentors, peers, and readers who continue to inspire me in the pursuit of knowledge and innovation.

Table of Content

I
Introduction To Nlp

Language is one of the most powerful tool that humanity possess. It is said that there are around 7000 languages spoken around the world. Early humans painted on cave walls, then developed spoken words to share stories and coordinate survival. Over thousands of years, languages grew more complex, leading to writing systems like cuneiform in Mesopotamia, hieroglyphs in Egypt, and alphabets in India and Greece.The printing press turned local words into global ideas, the telegraph and telephone collapsed distances that once divided continents, and the internet gave every voice the power to reach millions in seconds.

Today, we stand at another turning point Natural Language Processing (NLP). Instead of merely carrying our words farther or faster, technology now tries to understand them. Search engines predict intent, chatbots hold conversations, and translation tools erase language barriers in real time. You might have learned about the evolution of languages in your history text book but now in this Book you won't just learn how AI is shaping their future you'll be a part of driving that change. Natural Language Processing originated in the 1950s, when computers were still rare, room-sized equipment. Alan Turing, a mathematician and pioneer in computer technology, wrote the landmark paper "Computing Machinery and Intelligence"in 1950. In it, he asked the intriguing question, "Can machines think?".

Turing suggested a practical way to explore that question what we now call the Turing Test[1]. In the test, a human would communicate via text with two unseen entities: one human, one machine. If the human judge couldn't reliably tell which was which, the machine could be said to "think." Though Turing didn't separate this task from artificial intelligence as a whole, the test inherently involved interpreting and generating natural language the very essence of what we now call NLP. This idea sparked decades of curiosity By 1954, researchers at Georgetown University and IBM had conducted the first public machine translation experiment. Their IBM 701 computer translated 60 Rus-

[1]The Turing Test, proposed by Alan Turing in 1950, is a measure of a machine's ability to exhibit intelligent behavior indistinguishable from that of a human.

sian statements into English with a vocabulary of just 250 words and a few grammar rules. The press praised it as proof that fully automatic translation was on the horizon. While that optimism was premature, real-world language proved significantly more complex; it signalled the beginning of NLP as a discipline. Throughout the 1960s and 70s, experiments like ELIZA (1966) a simple chatbot that mimicked a therapist showed that even pattern-based language manipulation could feel intelligent. Though ELIZA didn't understand meaning, users often felt "understood," revealing how powerful even simple language technology could be.

Figura I.1: ELIZA chatbot interface

These early efforts taught researchers two enduring lessons:

1.) *Language is messy and nuanced, and cannot be fully captured by a few rules.*

2.) *Human communication and technology are deeply intertwined, and bridging them would require creativity across linguistics, computer science, and mathematics.*

From these humble origins, NLP progressed to rule-based systems, statistical methodologies in the 1990s, and today's deep learning models, such as transformers. However, Turing's audacious query sparked the idea that a machine could use language.

The Shift to Statistical Approaches and the Data Revolution

By the late 1970s and early 1980s, it was obvious that even the most complex rule-based systems could not capture the entire richness of human language. Every exception necessitated another rule, and each new realm revealed unanticipated ambiguity. Researchers began to investigate a different approach: statistical approaches.Instead of hard-coding grammar and vocabulary, they turned to probability and data. The concept was simple but effective: *rather than deliberately training a computer to "understand"English, why not allow it to learn patterns by analysing enormous amounts of genuine text?*

This change occurred simultaneously with two major influences that transformed the field. First, digitised text corpora became widely available, providing machines with the raw material required for statistical language learning. Simultaneously, computational power has continually increased, allowing for the processing and analysis of large datasets required for such approaches. Together, these advancements paved the path for powerful new tactics. Hidden Markov Models (HMMs) and n-gram language models quickly became statistical mainstays. Speech recognition was one of the most apparent triumphs, with systems such as IBM's Tangora demonstrating that statistical models could outperform the inflexible, rule-based architectures that dominated preceding decades.

Building on these breakthroughs, statistical methods transformed NLP throughout the late 1980s and 1990s. Researchers began to apply probabilistic models not just to speech recognition but also to tasks like part-of-speech tagging, syntactic parsing, and information retrieval. These methods allowed machines to handle the inherent ambiguity of human language more flexibly: instead of choosing a single "correct" interpretation, statistical models could weigh multiple possibilities and select the most probable one based on context.

This era also saw the development of large, annotated datasets such as the Penn Treebank, which became a benchmark for training and evaluating models. The availability of such resources created a virtuous cycle: better datasets led to better models, and better models spurred interest in building even larger datasets. By the mid-1990s, statistical machine translation (SMT) had begun to replace rule-based approaches. IBM's alignment models, for example, treated translation as a probabilistic problem learning word correspondences and phrase structures directly from bilingual text rather than relying on handcrafted dictionaries.

The rise of the World Wide Web amplified these trends by generating an unprecedented volume of textual data. Search engines leveraged statistical NLP for ranking algorithms, spell correction, and query expansion, making these techniques a foundational part of the emerging internet economy. This period marked a turning point: NLP had moved from a niche research endeavor into a practical technology with real-world applications, setting the stage for the machine learning revolution of the 2000s.

Deep Learning and the Transformer Revolution.

By the early 2010s NLP was ready for another leap forward. Traditional statistical methods had plateaued. While they performed well on structured tasks they struggled with the subtle meaning, context, and long-range dependencies that natural language often requires. Deep learning, which was already making waves in computer vision, offered a new way forward.Recurrent Neural Networks (RNNs) became the backbone of early deep learning approaches to NLP. Unlike previous models RNNs could process sequences of words step by step, maintaining a "memory" of what came before. This made them particularly effective for tasks like sentiment analysis, language modeling, and speech recognition. Yet RNNs had limitations. They struggled to retain information over long passages of text and were computationally expensive to train.

The introduction of Long Short-Term Memory (LSTM) networks in the late 1990s, which became popular in the 2010s, addressed some of these issues. LSTMs improved the handling of long-term dependencies, enabling breakthroughs in tasks like machine translation, most famously through systems like Google Translate, which began shifting from phrase-based statistical methods to neural approaches around 2016.The real turning point came in 2017 when a team at Google unveiled the Transformer architecture in the paper "Attention Is All You Need." Transformers abandoned recurrence altogether in favor of self-attention, a mechanism that allows models to look at all parts of a sentence simultaneously and decide which words are most relevant to each other. This design not only captured context more effectively but also enabled parallel computation, vastly accelerating training on large datasets.

The Transformer's impact was immediate and profound. Models like BERT (Bidirectional Encoder Representations from Transformers) in 2018 introduced a new way to pretrain language models on massive text corpora before fine-tuning them for specific tasks. BERT could grasp

context in both directions of a sentence, setting new records on benchmarks like the GLUE test for language understanding. Around the same time OpenAI's GPT series showed that scaling up Transformers could produce models capable of surprisingly fluent text generation, creative writing, and even basic reasoning. T5, or Text-to-Text Transfer Transformer, demonstrated the versatility of a unified approach by framing nearly every NLP task as text-to-text translation. These breakthroughs revolutionised not just academic benchmarks but real-world applications. Search engines became smarter at interpreting queries, virtual assistants like Siri and Alexa improved in conversational ability, and tools for summarisation, translation, and question answering became more accurate and accessible. NLP was no longer a niche research field. It had become a core technology shaping communication, business, education, and entertainment.

Perhaps most strikingly, Transformers made NLP models scalable. The larger the dataset and model, the more capabilities seemed to emerge. This phenomenon has led to today's large language models, which can draft emails, generate code, translate across dozens of languages, and even create poetry. The leap from early experiments like ELIZA to modern models demonstrates just how far the field has come in under a century.

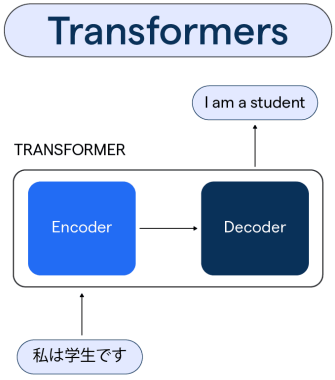


Figura I.2: Transformer Architecture

II
Text Representation and Preprocessing

Language is beautiful, but raw text is messy. Humans are remarkably good at reading between the lines.

The movie was **AMAZING!!! :)**

You instantly know it's positive, and the emoji only adds extra excitement. For a computer, however, this is just a string of characters with uppercase letters, exclamation marks, and symbols it doesn't inherently understand.Computers are logical. They don't "feel" words the way we do. If you give them a raw sentence, they won't know that amazing and great are similar, or that runs, ran, and running are all forms of the same verb. To bridge this gap, we need a way to clean, standardize, and structure text before feeding it into any model. This process is called text representation and preprocessing, and it is the foundation of every single NLP pipeline.Think of preprocessing as preparing ingredients before cooking. If you throw unwashed vegetables, peels, and dirt straight into the pan, the result won't be edible. The same goes for raw text without cleaning and preparation, the model won't be able to make sense of it.

What You'll Learn in This Chapter

By the end of this chapter, you'll be able to:

1. Understand why preprocessing is essential for NLP
2. Apply common text cleaning techniques
3. Break text into smaller pieces using tokenization
4. Remove unimportant words (stopwords)
5. Reduce words to their root form using stemming and lemmatization
6. Practice these concepts with hands-on Python examples

Let's get started.

Why Preprocessing is Important?

Preprocessing is a crucial step in Natural Language Processing because raw text is rarely clean or consistent. Real-world data often contains noise such as typos, extra punctuation, inconsistent capitalization, stop words, or irrelevant symbols that can confuse models. By standardizing and cleaning text, preprocessing improves the signal-to-noise ratio, making patterns easier for algorithms to detect.

Example:

- The movie was AMAZING!!!
- movie amazing

To a human, both sentences clearly express the same sentiment. However, to a computer, the first appears far more complex due to uppercase letters, punctuation, and extra words. Through preprocessing such as lowercasing, removing punctuation, and filtering unnecessary words both sentences are transformed into nearly identical representations, enabling the model to focus on meaning rather than formatting quirks. This reduction in complexity improves model performance and generalization, lowers computational costs by reducing vocabulary size and dimensionality, and ensures that feature extraction methods like TF-IDF[1] or embeddings[2] work more effectively. Proper preprocessing prevents models from overfitting to irrelevant details and increases the reliability of predictions. Ultimately, it makes NLP systems faster, more accurate, and more robust, turning messy, unstructured text into high-quality input suitable for tasks like sentiment analysis, classification, and machine translation.

[1] TF-IDF measures how important a word is within a document compared to a collection of documents.

[2] Embeddings are vector representations of words capturing their meaning and relationships.

Steps in Pre-Processing

Text preprocessing is the foundation of any NLP workflow, transforming raw, unstructured text into a clean and uniform format that models can effectively process. The process typically begins with *text cleaning*, where irrelevant elements such as punctuation, special characters, HTML tags, or URLs are removed to reduce noise.

Next, lowercasing is applied so that words like "Apple" and "apple" are treated as the same token unless case itself carries meaning, which helps avoid redundancy.

Tokenization then breaks the text into smaller units such as words or subwords, enabling the model to analyze each component separately. Stop word removal is often performed to discard common words like "the," "is," or "and," which generally add little value for many tasks, thereby reducing vocabulary size and focusing on meaningful terms.

Normalisation follows, using stemming or lemmatisation to reduce words to their base form stemming trims suffixes mechanically (for example, "running" becomes "run"), while lemmatisation uses linguistic rules to map a word to its proper root (for example, "better" becomes "good"). Handling negations is also essential because simply removing words like "not" can reverse sentiment; one approach is to merge the negation with the following word, as in converting "do not like" to "do-not like." In addition, rare or overly frequent words may need to be managed rare terms can be replaced with placeholders such as <UNK> to avoid noise, while frequent terms may be limited so they do not dominate the model's focus. Spelling correction and standardisation help address typos, slang, or abbreviations (for instance, transforming "Ths mvie ws gr8" into "This movie was great") to improve consistency.

Once text is cleaned and standardised, it must be converted into a numerical form through *vectorisation* or feature representation. Common approaches include Bag-of-Words, TF-IDF[1], and word embeddings[2] such as Word2Vec or GloVe. Depending on the task, advanced preprocessing steps can also be applied, like removing or converting emojis, expanding contractions (e.g., "don't" to "do not"), or applying part-of-speech tagging and dependency parsing for richer linguistic features. In practice, preprocessing choices should be tailored to the specific application: for example, in sentiment analysis, over-cleaning may strip important context such as exclamation points or emojis that signal strong emotions. Testing and fine-tuning different preprocessing configurations ensures the final pipeline is optimized for the task.

The `preprocess_text` function in next section demonstrates how raw text can be transformed into clean, consistent tokens ready for analysis. The next section will explore how these preprocessed tokens can be converted into numerical representations using techniques such as Bag-of-Words and TF-IDF, which form the foundation for most NLP models.

```python
import re
import nltk
import spacy
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from textblob import TextBlob
from collections import Counter
from sklearn.feature_extraction.text import
    CountVectorizer, TfidfVectorizer

# Download resources (run once)
nltk.download('punkt_tab', quiet=True) # Download
    punkt_tab
nltk.download('punkt')
nltk.download('stopwords')
nlp = spacy.load("en_core_web_sm")

def preprocess_text(text, keep_negations=True,
    use_lemmatization=True, min_freq=2):
    """
    Preprocess raw text into clean tokens for NLP
        tasks.
    """
    # 1. Cleaning
    text = re.sub(r"http\S+|www\S+", "", text)
    text = re.sub(r"<.*?>", "", text)
    text = re.sub(r"[^\w\s]", "", text)

    # 2. Lowercasing
    text = text.lower()

    # 3. Tokenization
    tokens = nltk.word_tokenize(text)

    # 4. Stopword removal
    stop_words = set(stopwords.words('english'))
    tokens = [w for w in tokens if w not in
        stop_words]

```

```
    # 5. Handle negations
    if keep_negations:
        result = []
        skip = False
        for i in range(len(tokens)-1):
            if tokens[i] == "not":
                result.append(tokens[i] + "_" +
                    tokens[i+1])
                skip = True
            elif skip:
                skip = False
            else:
                result.append(tokens[i])
        if not skip:
            result.append(tokens[-1])
        tokens = result

    # 6. Normalization
    if use_lemmatization:
        doc = nlp(" ".join(tokens))
        tokens = [token.lemma_ for token in doc]
    else:
        ps = PorterStemmer()
        tokens = [ps.stem(w) for w in tokens]

    # 7. Spelling correction
    tokens = [str(TextBlob(w).correct()) for w in
        tokens]

    # 8. Handle rare words
    freq = Counter(tokens)
    tokens = [w if freq[w] >= min_freq else
        "<UNK>" for w in tokens]

    return tokens

dummy_texts = [
    "The movie was AMAZING!!! I loved it.",
    "I did not like the movie at all.",
    "What an awesome film, truly enjoyed it."]
```

```
72
73 # Preprocess each document
74 preprocessed_docs = ["
       ".join(preprocess_text(doc)) for doc in
       dummy_texts]
75
76 # Bag-of-Words
77 bow_vectorizer = CountVectorizer()
78 bow_matrix =
       bow_vectorizer.fit_transform(preprocessed_docs)
79 print("Bag-of-Words feature names:",
       bow_vectorizer.get_feature_names_out())
80 print("Bag-of-Words matrix:\n",
       bow_matrix.toarray())
81
82 # TF-IDF
83 tfidf_vectorizer = TfidfVectorizer()
84 tfidf_matrix =
       tfidf_vectorizer.fit_transform(preprocessed_docs)
85 print("TF-IDF feature names:",
       tfidf_vectorizer.get_feature_names_out())
86 print("TF-IDF matrix:\n", tfidf_matrix.toarray())
```

III
Classical NLP Models and Techniques

Classical natural language processing methods laid the foundation for everything modern NLP achieves today. Before the rise of deep learning and large-scale language models, researchers relied on a combination of statistical models, heuristic rules, and carefully designed features to analyze and process text. These methods may seem simple compared to today's advanced architectures, but they played a crucial role in shaping the evolution of the field.Classical NLP methods are still highly relevant in many real-world scenarios. For example, when working with small datasets, where deep learning models struggle to generalize, statistical approaches often perform better. They are also useful in environments with limited computing resources since these methods require far less memory and processing power compared to modern deep learning systems. Another major strength lies in their interpretability: unlike neural networks, which often behave as black boxes, classical techniques allow researchers and practitioners to clearly understand why a particular decision was made. In this chapter, we will explore the foundations of these methods in depth. We begin with the Bag of Words model, one of the earliest and simplest approaches to text representation, which transforms text into numerical vectors based on word frequency. Building upon this, we will study n-grams, which capture word sequences and allow models to understand context beyond individual words. Next, we move to part-of-speech tagging, an essential task that assigns grammatical categories such as nouns, verbs, and adjectives to words in a sentence, enabling more sophisticated syntactic and semantic analysis. We will also introduce named entity recognition, a technique for identifying proper nouns and classifying them into categories such as names of people, organizations, or locations. Syntactic parsing will then be discussed as a way to uncover the structure of sentences and understand how words relate to each other. Closely related to parsing is chunking, a process that groups words into meaningful phrases, such as noun phrases or verb phrases, which can be directly useful for downstream applications.

After building a strong theoretical foundation, we will apply these techniques in practice by constructing a spam classifier. This hands-on project will show how classical NLP methods can work together in a real-world text classification problem. You will learn how to preprocess the data, extract features using Bag of Words and n-grams, enhance the feature set with linguistic information from part-of-speech tagging and named entities, and finally train a statistical classifier to detect unwanted spam messages.Along the way, we will also cover feature selection and dimensionality reduction. These two tools are essential for handling high-dimensional text data, which can easily contain tens of thousands of features. Feature selection helps identify the most informative features for the task, while dimensionality reduction transforms large feature spaces into compact representations without losing critical information. By the end of this chapter, you will not only understand the theory behind classical NLP methods but also gain practical experience in combining them to solve a complete text classification problem. These skills will serve as a stepping stone for more advanced NLP techniques, while also giving you reliable tools that remain effective in many practical applications.

Bag of Words and N Grams

The Bag of Words (BoW) model is one of the earliest and most influential approaches to representing text. In this method, each document is treated as a "bag" of its words, meaning that the order of words is ignored and only their occurrence is recorded. The representation is typically created by counting the frequency of each word in the document, which is then stored in a feature vector. Although this approach discards grammar and word order, it captures essential information about the vocabulary of a document and has proven surprisingly effective in many applications. For example, in tasks such as spam filtering or topic categorization, the frequency of certain keywords is often sufficient to distinguish between different classes. Words like "free," "win," or "offer" are strong indicators of spam, while terms such as "finance," "sports," or "technology" can help identify the subject matter of an article. Despite its simplicity, the Bag of Words model provides a powerful baseline for text classification tasks.

Formally, suppose we have a vocabulary $V = \{w_1, w_2, \ldots, w_{|V|}\}$ of size $|V|$. A document d can be represented as a vector

$$\mathbf{x}_d = [f(w_1, d),\ f(w_2, d),\ \ldots,\ f(w_{|V|}, d)],$$

where $f(w_i, d)$ is the frequency (or sometimes a binary indicator) of the word w_i in document d. This results in a high-dimensional vector space in which each dimension corresponds to one word in the vocabulary.

N Grams enhance the Bag of Words model by taking into account not only single words but also brief sequences of adjacent words. These sequences retain context and word order, which a basic BoW representation fails to preserve. For example, the bigram "New York" carries a distinct meaning that would go unrecognized if "New" and "York" were considered separate unigrams. Utilizing bigrams or trigrams allows the model to more effectively recognize local dependencies and frequently occurring phrases in the text.Mathematically, an n-gram is a sequence of n consecutive words from a document. If a sentence is represented as a sequence of tokens $T = (t_1, t_2, \ldots, t_m)$, then an n-gram is defined as

$$g_i = (t_i, t_{i+1}, \ldots, t_{i+n-1}),$$

for $1 \leq i \leq m - n + 1$. For example, for the sentence "I love natural language processing," the bigrams are:

$$(I, love),\ (love, natural),\ (natural, language),\ (language, processing)$$

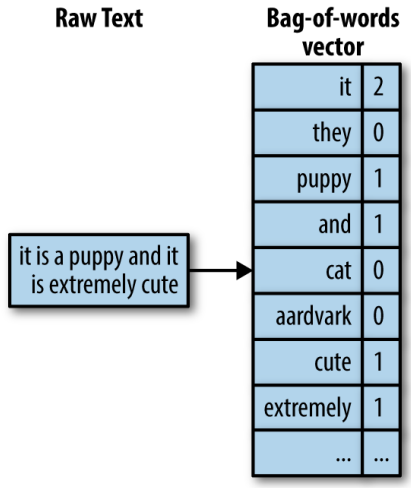


Figura III.1: Bag-of-words vector

While n-grams provide richer context, they also increase the size of the vocabulary dramatically. The maximum number of possible n-grams is approximately

$$|V|^n,$$

where $|V|$ is the vocabulary size. This exponential growth leads to very large and sparse feature matrices, which can be challenging to handle both computationally and statistically. Sparsity often results in overfitting, especially when the dataset is small.Therefore, professionals often test various n-gram ranges (such as unigrams combined with bigrams) instead of utilising extremely large n values. The objective is to find a middle ground between obtaining useful context and maintaining a feasible feature space. Through the use of Bag of Words and n-gram representations, you will start to grasp one of the key trade-offs in natural language processing: balancing the need to capture contextual meaning with the challenge of minimizing excessive sparsity. These methods establish the basis for more sophisticated approaches and remain essential tools in contemporary text analysis workflows.

Part-of-Speech Tagging

The technique of giving each word in a sentence a grammatical category is known as part-of-speech (POS) tagging. Familiar classes like noun, verb, adjective, adverb, pronoun, preposition, conjunction, and determiner are included in these categories. For many applications involving natural language processing, POS tagging offers a structured representation of language by labelling words in this manner.Accurate POS tagging greatly enhances downstream applications. For example, lemmatization relies on POS information to reduce words to their canonical or root form: the word running is lemmatized to run when identified as a verb, but remains running if it is tagged as a noun. Similarly, syntactic parsing uses POS tags to build parse trees that describe sentence structure, while named entity recognition benefits from knowing whether a token is functioning as a noun or something else in the given context. In this way, POS tagging acts as a bridge between raw text and more advanced linguistic analysis.

Prior to the introduction of statistical techniques like Hidden Markov Models, rule-based systems with manually constructed grammars were frequently employed in early POS tagging efforts. With their high levels of accuracy across all domains, machine learning and deep learning algorithms have largely taken over today. Nevertheless, POS tagging is now available without the need for algorithm design knowledge thanks to contemporary NLP frameworks like NLTK, spaCy, or Stanford CoreNLP. Processing sentences and seeing the grammatical tags given to each token only requires a few lines of code.A useful exercise is to tag a few sentences of your own writing and examine the results. Pay special attention to how the system resolves ambiguity. Many English words belong to more than one part of speech depending on context. The word book is a common example. In the sentence "I will book a ticket," the system correctly identifies book as a verb. In contrast, in the sentence "The book is on the table," the same word is tagged as a noun. This ability to disambiguate based on surrounding context highlights the strength of modern POS taggers.

Beyond its role in individual tasks, POS tagging remains a fundamental building block of NLP pipelines. Whether used for shallow parsing, feature extraction, or as input to machine learning models, POS information provides a layer of linguistic knowledge that purely statistical methods often lack. As such, understanding and experimenting with POS tagging is an important step for anyone learning about classical NLP methods.

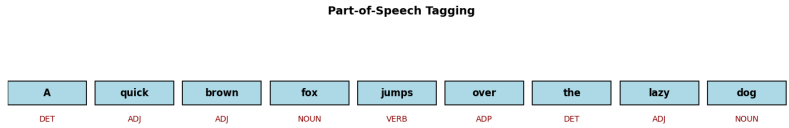


Figura III.2: POS Example

POS Implementation using Spacy

```
import spacy
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

# Load English model
nlp = spacy.load("en_core_web_sm")

# Input sentence
sentence = "A quick brown fox jumps over the lazy
    dog"
doc = nlp(sentence)

# Extract words and their POS tags
words = [token.text for token in doc]
pos_tags = [token.pos_ for token in doc]

# Create figure Using Matplotlib
plt.figure(figsize=(14, 3))
ax = plt.gca()
ax.set_xlim(0, len(words))
ax.set_ylim(-1, 2)
ax.axis("off")
```

```
22
23 # Draw each word with a box and its POS tag
24 for i, (word, pos) in enumerate(zip(words,
       pos_tags)):
25     # Draw rectangle for the word
26     rect = Rectangle((i, 0.2), 0.9, 0.5,
           facecolor="lightblue", edgecolor="black",
           linewidth=1.2)
27     ax.add_patch(rect)
28     # Add word inside the box
29     plt.text(i+0.45, 0.45, word, ha="center",
           va="center", fontsize=12, weight="bold")
30     # Add POS tag below the box
31     plt.text(i+0.45, -0.1, pos, ha="center",
           va="center", fontsize=10, color="darkred")
32
33 plt.title("Part-of-Speech Tagging", fontsize=14,
       weight="bold")
34 plt.tight_layout()
35 plt.show()
```

Named Entity Recognition

Named Entity Recognition (NER) is the task of automatically identifying and classifying references to real-world entities in text. These entities typically fall into categories such as people, organizations, geographic locations, dates, times, numerical values, products, and events. By highlighting these elements, NER helps transform unstructured text into structured information that can be used in a wide range of applications.

To illustrate, consider the sentence: "*NASA launched the Artemis I mission from Kennedy Space Center in 2022*." A robust Named Entity Recognition (NER) system would correctly classify NASA as an organization, Artemis I as a mission or event, Kennedy Space Center as a location, and 2022 as a date. This simple example demonstrates how raw text can be enriched with semantic labels, turning ordinary sentences into data that is far more useful for analysis.There are many different uses for NER. It enables systems to automatically extract facts from big document collections in information extraction. When building a knowledge graph, NER assists in locating the graph's nodes, which include individuals, institutions, and locations. These nodes can subsequently be connected by relationships. In question-answering systems, NER is also essential. For example, when asked, "Who founded Microsoft?"the system must identify and categorise named entities in order to find the right response. NER is also used in journalism and business intelligence to monitor mentions of leaders, businesses, and geopolitical events in large text streams.Historically, NER systems began with handcrafted rules and lexicons. Later, statistical models such as Hidden Markov Models and Conditional Random Fields became popular because they could generalize better across unseen data. Today, state-of-the-art NER relies heavily on deep learning models, especially those built on transformer architectures like BERT and its variants, which capture rich contextual information. These modern systems achieve impressive accuracy in standard benchmarks, but challenges remain.

One key challenge is dealing with ambiguity and variation. A word like

Apple could refer to a fruit or to a technology company, depending on context. Similarly, multi-word entities such as The New York Times or Bank of America must be recognized as single entities rather than separate tokens. Another difficulty arises when NER systems are applied to specialized domains such as biomedical text or legal documents, where the vocabulary and entity categories differ significantly from general newswire text. Even in multilingual contexts, systems may struggle due to differences in grammar, morphology, and orthography across languages.

Modern NLP libraries, including spaCy, NLTK, and Stanford CoreNLP, make it straightforward to experiment with NER. For instance, spaCy provides an interactive visualizer that highlights entities in color-coded categories, allowing users to see at a glance how text is being interpreted. Trying out NER on news headlines, social media posts, or your own writing offers valuable insight into both the strengths and limitations of current tools. You may observe that familiar and common names are tagged with high accuracy, while unusual or compound entities pose difficulties.

By experimenting with different texts and domains, you will begin to appreciate both the promise and the complexity of NER. Despite its challenges, NER remains one of the most practical and impactful tools in the NLP toolkit, bridging the gap between raw text and structured knowledge that can drive real-world applications.

Figura III.3: NER example

NER Implementation

```
# Install spaCy if not already installed
# !pip install spacy

import spacy
from spacy import displacy

# Load English model
nlp = spacy.load("en_core_web_sm")

# Example text
text = "ISRO launched Chandrayaan-3 from the
    Satish Dhawan Space Centre in 2023"

doc = nlp(text)

# Print detected entities with labels
print("Named Entities, Phrases, and Labels:")
for ent in doc.ents:
    print(f"{ent.text} -> {ent.label_}")

# Visualize entities in a Jupyter Notebook
displacy.render(doc, style="ent", jupyter=True)
```

Syntactic Parsing and Chunking

While part-of-speech tagging assigns grammatical categories to individual words, syntactic parsing goes a step further by uncovering the relationships between them.It makes an effort to respond to queries like "Which word is the sentence's subject?"What is the primary verb? In a phrase, which terms belong together? As a result, parsing gives sentences a structural representation that reflects the logic and syntax of natural language.

There are two major approaches to syntactic parsing. **Dependency parsing** models sentences as directed graphs, where words are nodes and grammatical relationships (such as subject, object, or modifier) are edges connecting them. For example, in the sentence The student read the book, the word read is identified as the root verb, student is linked as its subject, and book as its object. Dependency trees are widely used in modern NLP systems because they provide compact, informative structures that are directly useful for downstream applications.

In contrast, **constituency parsing** represents sentences as nested phrase structures, often visualized as trees. It groups words into phrases such as noun phrases (the big red car), verb phrases (is running quickly), and prepositional phrases (in the park). Constituency trees capture the hierarchical nature of language and reflect the syntactic theories of traditional linguistics. They are especially valuable in applications like machine translation, question answering, and formal grammar checking, where phrase boundaries play an important role. Between full parsing and simple tagging lies chunking, sometimes called shallow parsing. Instead of producing complete parse trees, chunking identifies and segments multi-word units such as noun phrases (a large dataset) or verb phrases (has been completed). This makes chunking faster and less computationally expensive while still providing valuable structure. For example, in information extraction, detecting noun phrases is often sufficient to identify entities and their attributes without requiring a full syntactic analysis.

Visualizing dependency or constituency trees for complex sentences can be an enlightening exercise. By examining these structures, you see how parsing helps machines recognize subject–verb–object relationships, modifiers, and

clause boundaries. This structural awareness enables practical applications such as relation extraction, sentiment analysis at the phrase level, question answering, and even grammar correction. Importantly, these techniques remain useful even without deep learning models. In low-resource settings or when interpretability is important, classical parsing and chunking methods offer a balance between linguistic insight and computational efficiency.

Syntactic Parsing and Chunking Using Spacy

```
import spacy
from spacy import displacy

# Load English model
nlp = spacy.load("en_core_web_sm")

# Example sentence
text = "The quick brown fox jumps over the lazy dog near the river bank."
doc = nlp(text)

# Dependency Parsing
print("Dependency Parsing (Head -> Dependent):")
for token in doc:
    print(f"{token.text:10} ({token.dep_}) --> {token.head.text}")

# Visualize dependency tree (works in Jupyter/Streamlit)
displacy.render(doc, style="dep", jupyter=True)

# --- Chunking (Noun Phrases) ---
print("\nNoun Phrases (Chunks):")
for chunk in doc.noun_chunks:
    print(chunk.text)

\newpage
```

```
Dependency Parsing (Head -> Dependent):
The         (det) --> fox
quick       (amod) --> fox
brown       (amod) --> fox
fox         (nsubj) --> jumps
jumps       (ROOT) --> jumps
over        (prep) --> jumps
the         (det) --> dog
lazy        (amod) --> dog
dog         (pobj) --> over
near        (prep) --> dog
the         (det) --> bank
river       (compound) --> bank
bank        (pobj) --> near
.           (punct) --> jumps
```

Figura III.4: Syntactic Parsing and Chunking

Label	**Full Form**	**Example from Sentence**
det	Determiner	The → fox
amod	Adjectival Modifier	quick → fox
nsubj	Nominal Subject	fox → jumps
ROOT	Root of the sentence	jumps → jumps
prep	Prepositional Modifier	over → jumps
pobj	Object of a Preposition	dog → over
compound	Compound	river → bank
punct	Punctuation	. → jumps

Tabela III.1: Dependency parsing labels and their full forms

Now that we have seen how to extract a wide range of features from word counts and ngrams to part of speech tags, entities, and syntactic structures, the next challenge is managing the explosive growth in dimensionality.This is where feature selection and dimensionality reduction techniques become essential.

Feature Selection and Dimensionality Reduction

Working with natural language data often results in very high-dimensional feature spaces when text is represented numerically. As we saw with Bag of Words (BoW) and N-grams, the vocabulary of even a modest text corpus can easily contain tens of thousands of unique words or n-grams.Each of of these turns into a possible feature, producing sparse and incredibly massive vectors.

This section is among the most crucial in the book since it tackles a problem that lies at the core of natural language processing. What happens when we scale is the true test, even if up to this point we have concentrated on methods to extract usable information from text – words, sentences, and grammatical structures The richness of language means that the number of possible features can explode very quickly. While on the surface having more features may appear advantageous (since we are capturing more details), in practice it introduces two significant and often crippling challenges:

1.) The Curse of Dimensionality:

We encounter what is known as the "curse of dimensionality"when the number of features increases, which is frequently far greater than the number of real data samples. This simply means that there is too much data for the model to consider, but not enough solid evidence to draw conclusions from.

Think of it like studying for an exam with a book that is 10,000 pages long, but you only get to read 50 of those pages. With so little exposure compared to the size of the material, you're more likely to memorize a few details from those 50 pages rather than truly understand the subject. When the exam comes, you'll struggle because the questions will be from the unseen 9,950 pages.

That is exactly what happens to machine learning models when we use methods like Bag of Words or N-grams without any control.Our dataset could only have a few thousand phrases, but the feature space grows to tens of thousands of terms or word combinations.

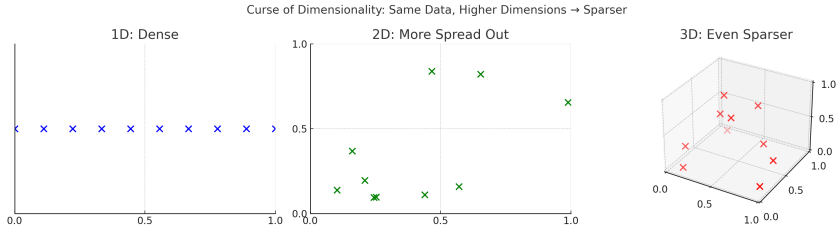


Figura III.5: The Curse of Dimensionality

1D (Line): 10 points cover the line densely.
2D (Square): The same 10 points look more spread out.
3D (Cube): Now those 10 points are scattered in a whole volume, making the space feel even emptier.

The model latches onto meaningless patterns as a result of the imbalance. For example, the model may mistakenly learn that any sentence that contains the word "spectacular"is positive if it only occurs once in a positive review.
This is called overfitting: the model performs well on the training data because it memorises these quirks, but when it sees new, unseen text, it fails miserably. Instead of capturing the essence of language, it becomes a noisy dictionary of coincidences. In essence, the curse of dimensionality shows us that “more features” is not always “better.” Beyond a point, too many features actually make the problem harder, not easier.

2.) Computational Inefficiency

High-dimensional data confuses models and drains computational resources. The greater the feature area, the more expensive it is to train, store, and process. Models take longer to converge (sometimes not at all), consume more memory, and can quickly become unsuitable for real-world or large-scale applications.Think of it like trying to run with a backpack stuffed with thousands of unnecessary items.

You could still move forward, but every step is heavier, slower, and more exhausting than it needs to be. Similarly, when our feature space explodes into tens of thousands of dimensions, the model carries around "extra baggage" that does little to improve performance but greatly increases the cost.
This inefficiency isn't just a minor inconvenience. In industrial-scale NLP tasks where datasets contain millions of documents, training times could stretch from hours into days, memory requirements could exceed hardware limits, and even storing the feature matrices can become a problem.

To overcome this challenge, researchers and practitioners typically rely on two broad strategies:

- **Feature Selection** – picking only the most informative features, like cleaning up the clutter and keeping just the essentials.
- **Dimensionality Reduction** – transforming the original high-dimensional data into a smaller, more compact representation, while still preserving as much meaning as possible.

Both approaches aim to lighten the "backpack" the model carries but they do so in fundamentally different ways. Let's explore.

Feature Selection

Feature selection is the process of finding and retaining only the most informative characteristics from an initial dataset. Rather than throwing every single word, sentence, or n-gram into the model, we painstakingly remove the useless or superfluous ones. You may think of it as clearing a cluttered toolbox: while it's tempting to keep every screwdriver, wrench, and bolt you come across, all you really need is a smaller set of the most effective tools for the job.

Why is this important? Because fewer features mean:

Less noise → the model doesn't waste time on irrelevant patterns.
Faster training → Smaller datasets are easier to compute.
Better generalisation → the model focuses on meaningful signals, not random quirks. Feature selection is one of the most practical remedies to the curse of dimensionality, as we recently described. It ensures that, rather than drowning in a sea of features, the model learns to swim with the correct strokes.

Popular Feature Selection Approaches

1. Filter Methods
These techniques score and rank features using statistical tests. Only the top-ranked features are retained, making this method fast and easy to apply. Common techniques include:

- **Chi-Square Test** – measures how strongly a word is associated with a class label.
- **Mutual Information** – quantifies how much knowing the presence of a word reduces uncertainty about the class.
- **TF-IDF (Term Frequency–Inverse Document Frequency)** – emphasizes words that are frequent in a document but rare across the corpus, often making them strong indicators for classification.

2. Wrapper Methods
These methods actually train and evaluate models on different subsets of features to find the most effective combination. While they are more computationally expensive, they can yield highly optimized results. Common approaches include:

- **Forward Selection** – start with no features and add them one by one if they improve performance.
- **Backward Elimination** – start with all features and remove the least useful ones step by step.
- **Recursive Feature Elimination (RFE)** – repeatedly build models, ranking and removing the weakest features until the optimal set remains.

3. Embedded Methods
Here, feature selection is baked into the model training process itself. These methods strike a balance between the speed of filters and the accuracy of wrappers. Examples include:

- **L1-Regularization (Lasso)** – naturally drives less important feature weights toward zero, effectively discarding them.
- **Decision Trees and Ensembles (e.g., Random Forests, Gradient Boosted Trees)** – rank features by how much they improve the purity of splits, providing a built-in measure of feature importance.

Dimensionality Reduction

While feature selection focuses on discarding irrelevant or redundant features, dimensionality reduction takes a different approach: it transforms the original features into a new, smaller set of dimensions. Instead of removing features altogether, it creates compressed representations that capture most of the important information in the data, A Simple way to picture this is with the toolbox analogy. If feature selection is like discarding tools you don't need, dimensionality reduction is like melting all your tools down and forging a new, smaller set of multipurpose master tools. These new tools may not look like the originals, but they retain their usefulness while being easier to carry around.

Principal Component Analysis (PCA) is the most often used dimensionality reduction technique. PCA identifies new axes, known as principle components, in the high-dimensional space that capture the most variance in data. The first principal component explains the most variance, followed by the second (which is orthogonal to the first), and so on. By projecting the data onto only the top few principal components, we may dramatically reduce dimensionality while maintaining the majority of the original variance. For example, a dataset with 10,000 features might be compressed into just 200 main components with just a minor loss of information.

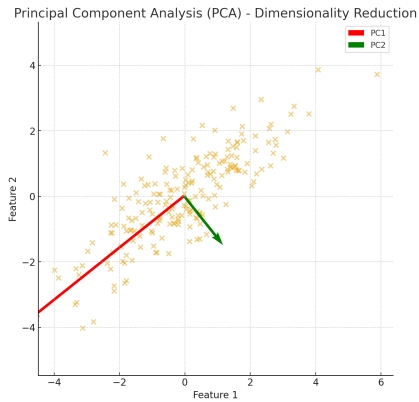


Figura III.6: PCA

In practice, dimensionality reduction is especially valuable when dealing with dense embeddings like word2vec, GloVe, BERT, or sentence transformers. Even though these embeddings are much smaller than raw BoW vectors, they can still be hundreds of dimensions long. Reducing them to 20–50 dimensions often speeds up downstream tasks like clustering or classification while making visualization easier.

How They Work Together

Feature selection and dimensionality reduction should not be viewed as competing procedures, but rather as complementing tactics that, when coupled, enable us to manage the overwhelming complexity of high-dimensional text data. Each serves a separate purpose: selection sharpens the emphasis by selecting the most important qualities, whereas reduction compresses information into compact but strong representations.

III.0.1 How Feature Selection and Dimensionality Reduction Work Together

Feature selection and dimensionality reduction are not competing approaches but rather *complementary strategies* that, when combined, help us manage the overwhelming complexity of high-dimensional text data. Each plays a distinct role: selection sharpens the focus by identifying the most meaningful features, while reduction compresses information into compact yet powerful representations.

A useful analogy is preparing for a journey. Feature selection is like choosing only the items you truly need from your suitcase, while dimensionality reduction is rolling or folding those items efficiently so that they fit neatly into a small backpack. The objective is to travel light but still be well-prepared.

A Typical Workflow in NLP

In practice, the two techniques often appear together in the following pipeline:

1. **Preprocessing:** Clean and normalize text (tokenization, lemmatization, stopword removal).
2. **Feature Extraction:** Transform text into numerical form. Examples include:
    - Bag-of-Words (BoW) with unigrams and bigrams,
    - TF-IDF weighted features,
    - Dense embeddings (e.g., word2vec, GloVe, BERT).
3. **Feature Selection:** Remove irrelevant or redundant features:
    - Drop high-frequency stop words that carry little discriminative power.
    - Use chi-square or mutual information to identify terms most correlated with the target.
    - Apply L1 regularization in a logistic regression model to shrink uninformative coefficients.
4. **Dimensionality Reduction:** Compress the remaining features into a compact representation:
    - PCA or LSA to project onto a smaller feature space,
    - Nonlinear methods like t-SNE or UMAP for visualization,

- Autoencoders for learned compressed embeddings.

Example: From 50,000 original BoW features → down to 5,000 selected terms → reduced to 200 principal components.

5. **Model Training:** Train the classifier on the optimized features, gaining both speed and robustness to overfitting.

6. **Evaluation and Iteration:** Test different levels of selection and reduction, compare accuracy, F1-score, and runtime, then strike a balance between predictive power and computational cost.

Real-World Example

Consider building a fake news detection system. A raw Bag-of-Words representation might yield 120,000 features. After feature selection, only 10,000 highly predictive terms remain (e.g., *breaking*, *exclusive*, *click here*). Applying PCA compresses these into 300 dimensions. Training a classifier on this representation is faster, less memory-intensive, and more generalizable than working with the raw feature set.

IV

About the Author

Henilsinh Raj is a researcher, author, and entrepreneur. As the founder of Axamine AI, he is building innovative solutions that make medical diagnosis faster, smarter, and more accessible. Passionate about simplifying complex technologies, he continues to inspire students, self-learners, and professionals to explore the world of AI with curiosity and confidence.